Sound Level Meter Project

1. Introduction

In this project you will construct and test a simple sound level meter circuit. This device is similar to that used in various pieces of audio equipment, and the level of sound is displayed using LEDs. There are a number of areas where you need to do some design work in order to calculate the component values. Although the project is to be constructed on a printed circuit board (PCB), you may wish to first try out the microphone amplifier and band pass filter sub-circuits on protoboard / breadboard.

2. General Circuit Description

Please refer to the overall circuit diagram on the next page when reading the following information. The sound level meter to be constructed is a combination of analogue and digital circuit functions. The sound is measured with a small microphone (M1) and its amplitude determines the number of LEDs which are illuminated. As the microphone produces a relatively small signal, amplification is required, and this is achieved using the microphone amplifier circuit based on transistor T1.

Audible sound occupies a frequency range of approximately 20 Hz to 16 kHz. The sound level meter that is going to be constructed is designed to only measure sounds in a reduced frequency range, and this is determined the band pass filter based on op-amp U1. To measure the signal amplitude, the signal is then rectified using components, D1, R11, and U2. The rectified signal is then passed through a low pass filter based on U3. The process of rectification and low pass filtering produces a DC signal which is proportional to the sound signal amplitude, and this is similar to the amplitude demodulation used in radio circuits.

The final step is to illuminate the LEDs according to this DC level. A PIC microcontroller is used to do this. The PIC takes in the DC signal on one of its analogue-to-digital (ADC) inputs, and via a suitable program, switches the appropriate number of LEDs using its digital outputs.

IMPORTANT: Where you see **D+** it is suggested that you demonstrate your working circuit to a member of staff or a demonstrator. It is much easier to test / fault find individual parts of the circuit rather than a fully constructed system.

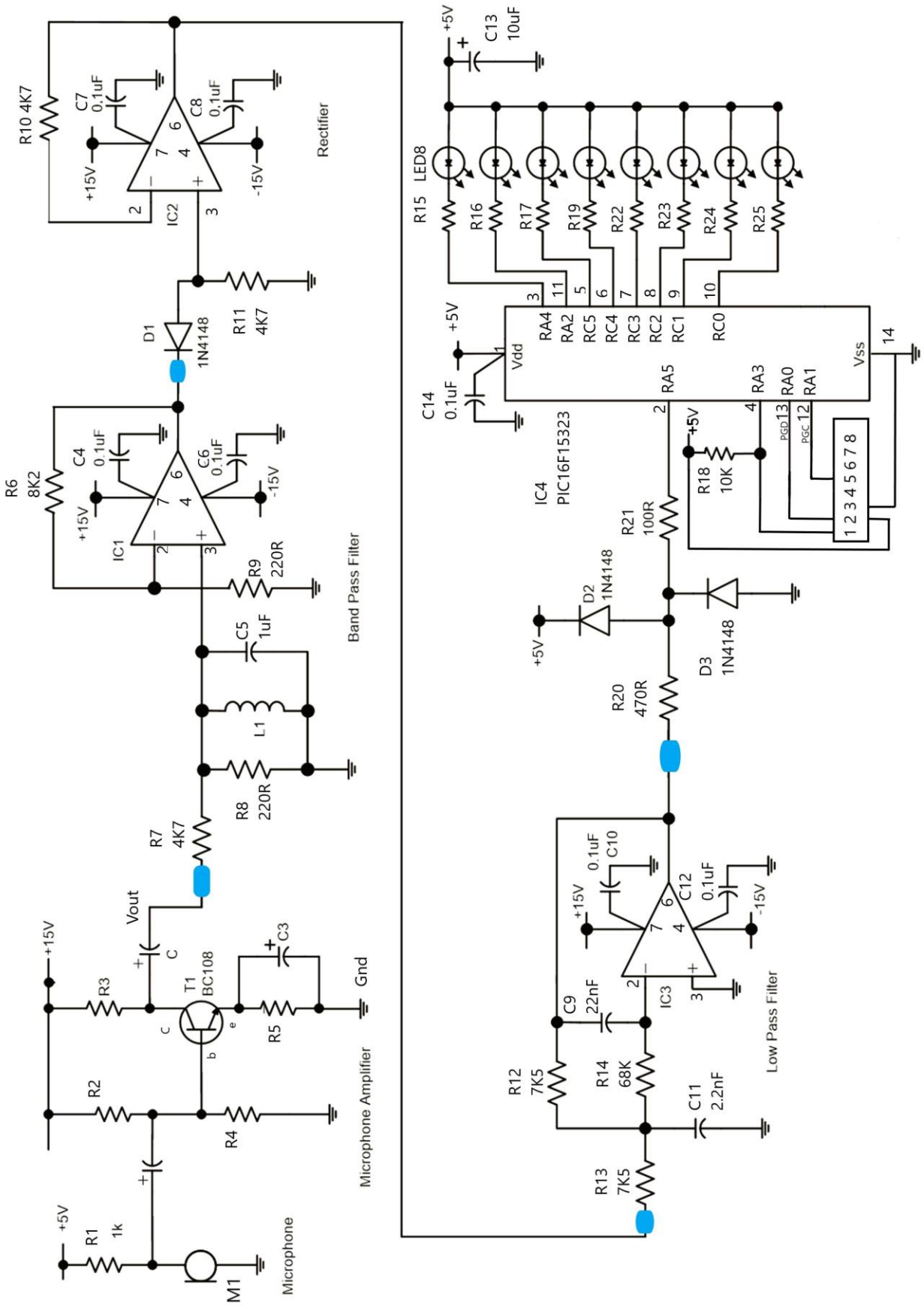


Figure 1: Overall Sound Level Meter Circuit Diagram

3. Microphone Amplifier Design

This amplifier circuit is based on the common emitter transistor amplifier, and this will have been covered in your previous analogue electronics lecture course notes.

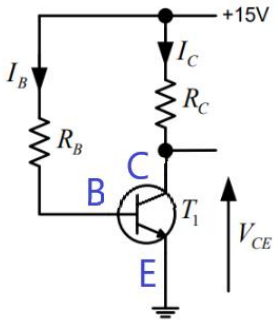


Figure 2: Circuit to determine the current gain

The transistor for the circuit is a BC108, and the pin-out is shown in Fig. 3 below:

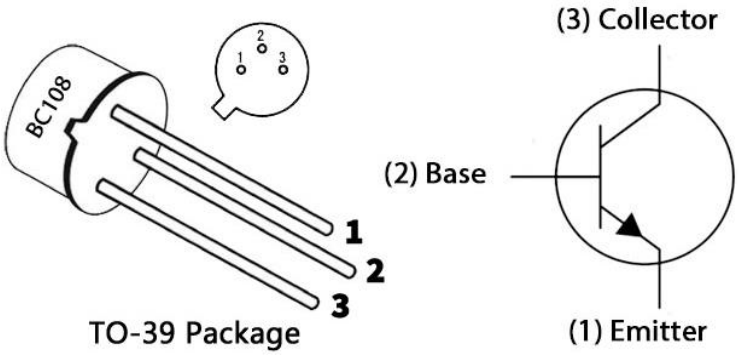


Figure 3: BC108 Pin-out

You first need to measure the current gain (β) of your transistor (T_1) as there is a wide range of possible gains due to manufacturing tolerances in this parameter. The range can be between 110 and 800. In this circuit, it is recommended that you operate T_1 with a quiescent collector voltage of approximately 7.5 V and a collector current (I_C) of approximately 2 mA. This can be achieved by setting R_c in Fig. 2 to 3k9. **Construct this circuit on protoboard / breadboard** and find a suitable value for R_B which will result in the collector voltage (V_{CE}) being approximately 7.5 V. You are likely to require a value in the MΩ range for R_B since the base current will be very small. Start with a 1MΩ resistor from the 0.25 W selection available in the stores. If you have never used protoboard / breadboard before, there are some guidance notes on Canvas.

Note: when using protoboard / breadboard use solid core wire (not multi-strand) as this is much easier to use.

You can use the following equations and table to help you find the most appropriate value for R_B and then β.

The base and collector currents can be calculated using:

$$I_B = \frac{15 - V_{BE}}{R_B} \quad (1)$$

$$I_C = \frac{15 - V_{CE}}{R_C} \quad (2)$$

Where V_{BE} is taken as 0.7 V for the silicon transistor.

The current gain can then be then calculated using:

$$\beta = \frac{I_C}{I_B} \quad (3)$$

$V_{BE} = 0.675$

R_B Ω	Calculated I_B mA	Measured V_{CE} V	Calculated I_C mA	Calculated β
1M	0.014	0.172	3.802	271.57
2.2M	0.0065	6.542	2.168	333.65
✓ 2.7M	0.0053	7.945	1.80	339.62

NOTE: It is important that you do not mix up your transistor with another one as they all have different values for β. If you do mix up / lose your transistor you will have to redo all of the above testing.

D+

You will need to keep the 3k9 resistor and use it for R3 in the microphone amplifier shown in Fig. 4.

A value for R5 needs to be calculated so the emitter voltage drop is approximately 10% of the supply voltage, i.e., $V_E = 0.1 \times 15\ V = 1.5\ V$. It can also be assumed that the emitter current is approximately equal to the collector current. After calculation (you will need to think of the appropriate equation), select the nearest resistor from the 0.25 W selection available in the stores.

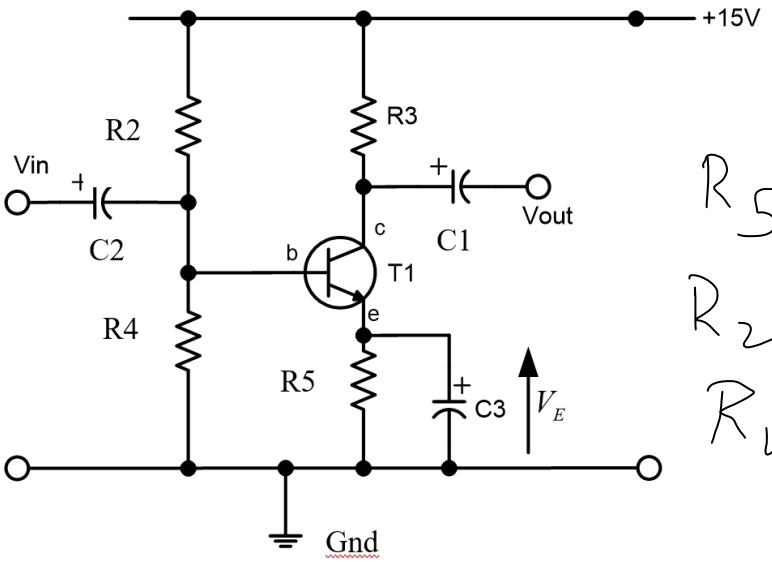


Figure 4: Microphone Amplifier Circuit

The current flow through the resistor chain created by R2 and R4 can be assumed to be $10 \times I_B$. Taking the voltage drop across R4 to be $V_{BE} + V_E$, calculate a value for the bias resistor R4. The remaining bias resistor R2 can now be calculated from:

$$R_2 = \frac{15 - (V_{BE} + V_E)}{10 \times I_B} \qquad (4)$$

Again, you will need to use the nearest resistor value from the stores.

To check that this design produces suitable bias levels, quickly construct the circuit in Fig. 4 on protoboard / breadboard and connect it to a 15 V bench power supply. You do not need to include capacitors C1, C2, and C3 at this stage.

Measure the collector, emitter, and base terminal voltages with respect to 0 V and record the values. The collector voltage should be approximately 7.5 V (i.e. half the supply voltage). Due to the use of the standard 0.25 W resistor values which have a 5 % tolerance, the collector voltage may not be exactly 7.5 V. However, provided it is within the range of 6 V to 10 V, then the circuit should operate correctly. If your collector voltage is outside this range, then double check your calculations.

Now add the capacitors. C1 and C2 are both $10\mu F$ and C3 is either $220\mu F$ or $330\mu F$ (one of these will be in the parts kit provided).

NOTE: These capacitors are electrolytic and therefore the polarity is important! If you do not understand the [polarity of these, please ask a demonstrator.

You are now required to check the AC voltage gain of this circuit. Using a signal generator, apply a 20-mV peak to peak, 1 kHz sinewave signal to terminal Vin. Monitor Vin and Vout on the oscilloscope and determine the AC voltage gain of the microphone amplifier. Make a note of the AC gain.

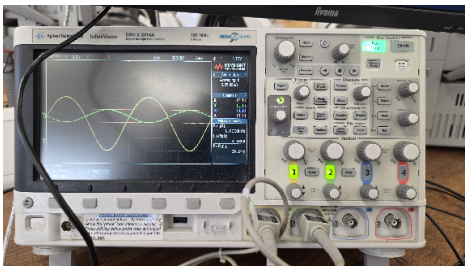
D+

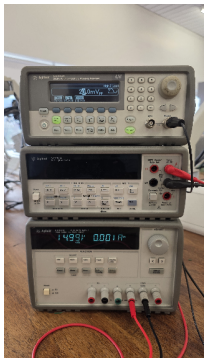
You can now start soldering your components into the PCB. The resistor in series with the microphone (R1) is 1kΩ.

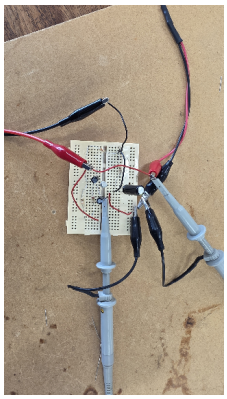
$V_{in} = 20mV$ (p-p)

$V_{out} = 4.18V$

$A = 209$







4. Bandpass Filter Design

The bandpass filter shown in Fig. 5 is used to limit the frequency range of the sound signal.

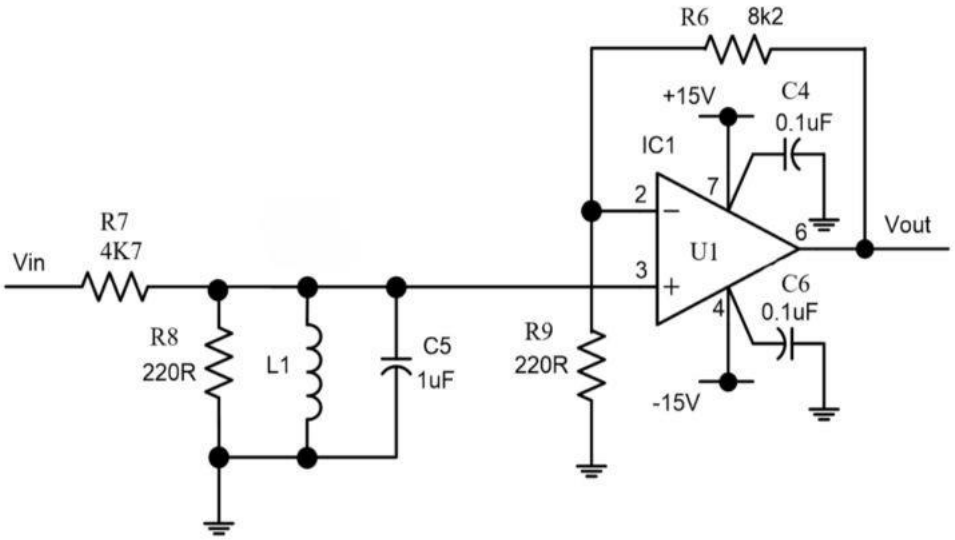


Figure 5: Band Pass Filter Circuit

This limitation is achieved using a parallel resonant circuit comprised of inductance L1 and capacitor C5. The resonant circuit is followed by an op-amp based voltage amplifier to produce a low impedance drive to the next stage. The resonant frequency should be set to 1.5 kHz, and it is suggested that you use a $1\mu F$ capacitor for C5. You now need to calculate the inductance value. The frequency of a resonant circuit is given by the equation below, use this to calculate a suitable inductance value.

$$f = \frac{1}{2\pi\sqrt{LC}} \quad (5)$$

$L = 11.30mH$

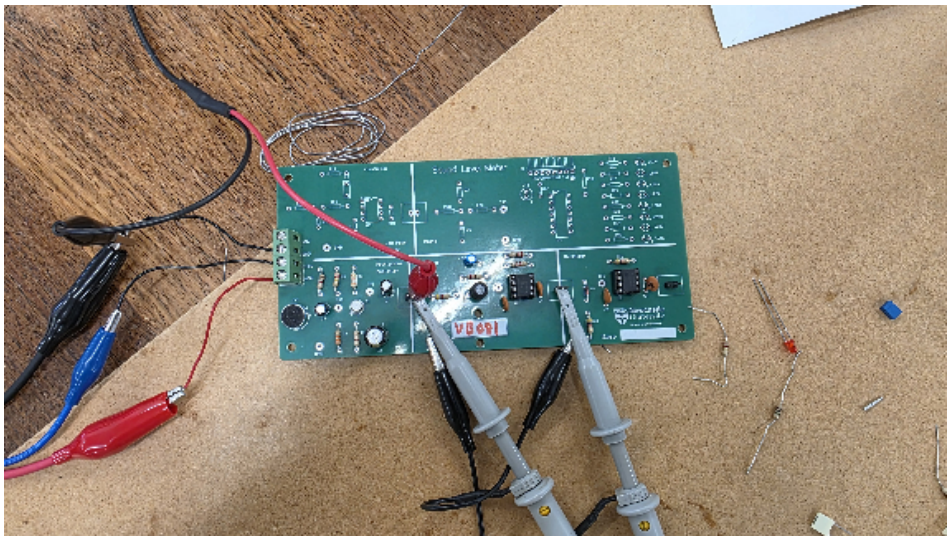
$\hookrightarrow 10mH$ //

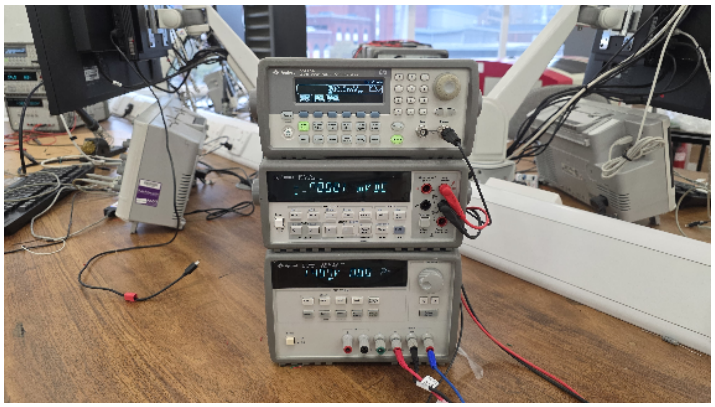
You can obtain this inductance value in the stores.

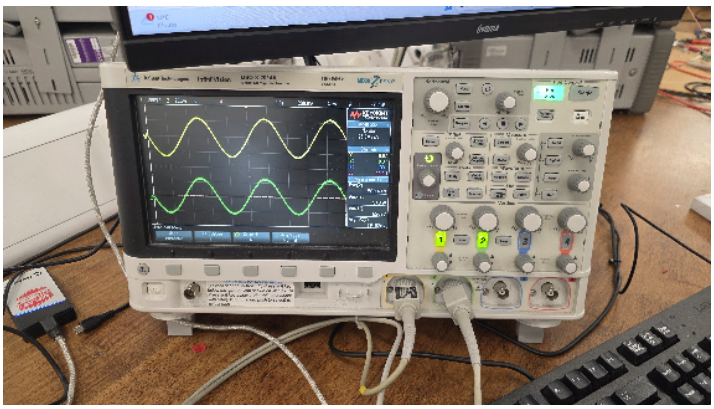
Now construct the band pass filter circuit shown in Fig. 5 on the provided printed circuit board. To test this circuit, you will need to connect it up to a +/- 15 V power supply. To check the operation of the circuit, you will need to apply a 1 V peak sinusoidal signal to Vin and monitor Vin and Vout on the oscilloscope. By varying the frequency of the signal, determine the frequency at which the filter produces the maximum gain. This is the resonant frequency. Record the magnitudes of Vin and Vout for a number of different frequencies between 500 Hz and 10 kHz. Enter these into a programme such as Excel or Matlab and plot the frequency response. You may need to take extra test data once you have plotted the response to get an acceptable curve. **Note:** connect the signal generator directly to Vin, do not pass the signal through the previous amplifier circuit.

Sinusoidal Signal Frequency	Measured Vin (P-P)	Measured Vout
500 Hz	0.5	0.28
1000		0.37
1500		0.48
1700		0.5
2000		0.47
5000		0.32
7000		0.28
9000		0.14
10000		0.12

D+







Calculations to Design Own Inductor

If you wish you can also design your own inductor based around a small magnetic ferrite ring.

The reluctance of a ring is given by:

$$S = \frac{l}{\mu \times A} \tag{6}$$

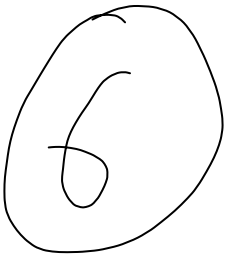
Where μ is the permeability, l is the mean length of the magnetic circuit and A is the effective cross-sectional area.

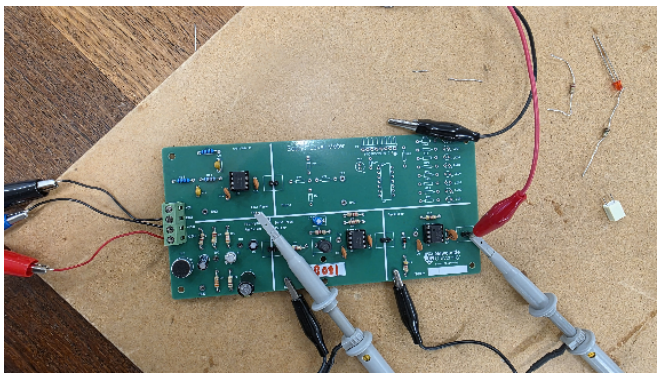
The inductance is given by:

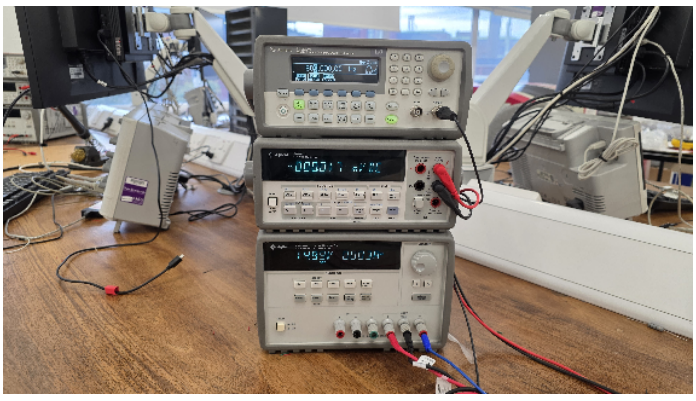
$$L = \frac{N^2}{S} \tag{7}$$

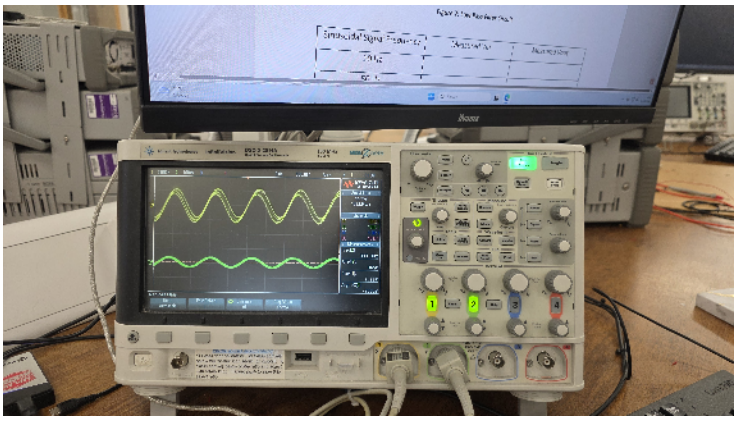
Using the above expressions, the number of turns N necessary to produce the required inductance can be calculated.

The ring material has a permeability of $\mu = 623 \times 10^{-5}\ \mathrm{H/m}$, the mean length of the ring is $0.054\ \mathrm{m}$. and the cross-sectional area is $0.262 \times 10^{-4}\ \mathrm{m}^2$.









5. Signal Rectifier

The signal rectifier circuit shown in Fig. 6 is used to remove the positive half of the signal produced by the band pass filter, and this is part of the process of detecting the signal amplitude. The signal diode D1 will only allow negative polarity current to pass down through R11. The voltage appearing across R11 is then applied to a unity gain amplifier (voltage buffer) to give the resulting signal a low impedance drive to the next stage.

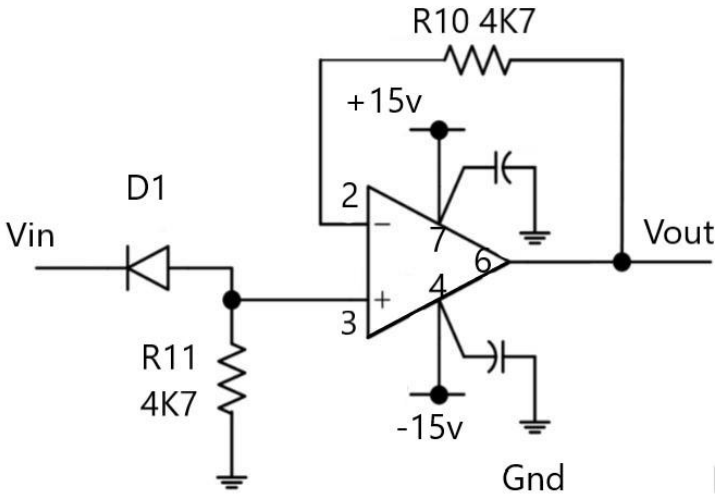
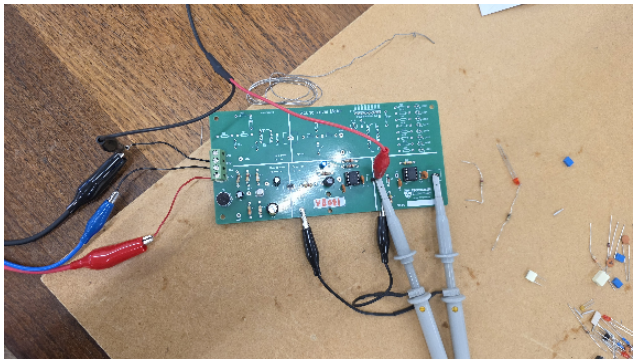


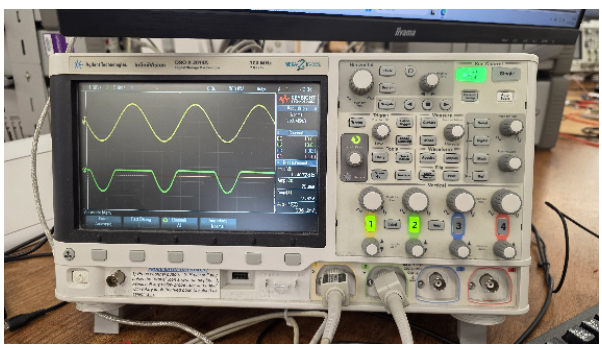
Figure 6: Rectifier Circuit

Again, this circuit requires a dual 15 V supply. Apply a 5 V peak, 1.5 kHz test signal to Vin and monitor Vin and Vout on the oscilloscope and record the waveforms. **Note:** connect the signal generator directly to Vin, do not pass the signal through the previous amplifier or band pass filter circuits.

D+







6. Low Pass Filter

The low pass filter smoothes the rectified signal to produce a DC value proportional to the input signal amplitude. The low pass design shown in Fig. 7 has quite a complex formula for determining the component values and therefore the complete design is given. After constructing the low pass filter circuit, apply a 1 V peak to peak sinusoidal signal to Vin and monitor Vin and Vout on the oscilloscope. Determine the circuit gain for a range of frequencies between 10 Hz and 1 kHz and plot a gain versus frequency graph using Excel or Matlab. Save this graph and use this to estimate the frequency at which the gain is 0.707.

Note: connect the signal generator directly to Vin, do not pass the signal through the previous circuits.

$f_c \simeq 900\,KHz$

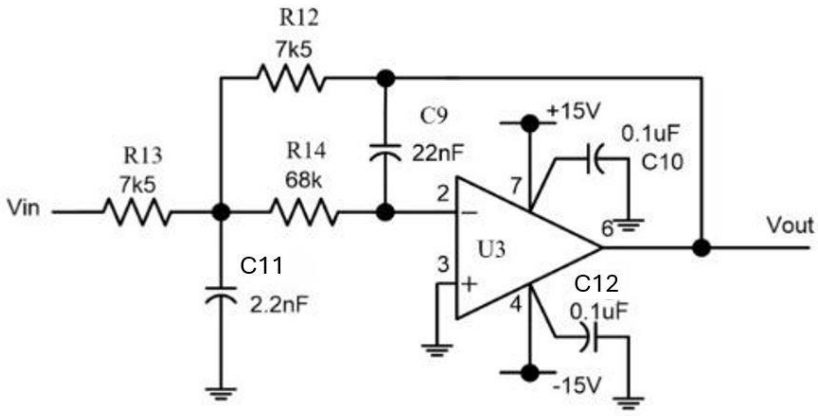


Figure 7: Low Pass Filter Circuit

Sinusoidal Signal Frequency	Measured Vin (p) (v)	Measured Vout (p) (v)
10 Hz	1	1
50 Hz	1	1
100 Hz	1	1
200 Hz	1	1
300	1	1
500	1	1
600	1	1
800	1	1
900	1	0.7
1000	1	0.6

D+

7. PIC Microcontroller

In this project, the function of the PIC microcontroller shown in Fig. 8 is to illuminate a number of LEDs depending on the magnitude of the DC voltage applied to Vin. The microcontroller chip is a PIC16F15323 which is manufactured by a company called Microchip (www.microchip.com). You will need to programme the device using C language but first some explanation of the hardware.

The microcontroller is set up to have 8 digital outputs RC0-RC5, RA2, and RA4 which are used to drive the LEDs. The microcontroller also has a number of analogue inputs, of which RA5 is used for this application. The network comprising R20, R21, D2, and D3 is used to protect the analogue input from damage due to accidental excessive voltage. If Vin is greater than 5.7 V, then diode D2 conducts and clamps the input voltage to this level. If Vin is less than -0.7 V, diode D3 clamps the input voltage to this level.

The decoupling capacitor C13 is included to act as a transient source of current for the LEDs as they are switched on and off. This helps to prevent interference in the circuit.

The PIC microcontroller is a very flexible device and the function of many of its pins need to be programmed as digital input, digital output, or analogue input. Also, various timing details such as the chip clock frequency needs to be set up.

There is quite a lot of detailed programme code associated with setting up the PIC, so to save you time we have provided the full working code which you just have to upload to the PIC. A copy of the programme is given in Appendix 1.

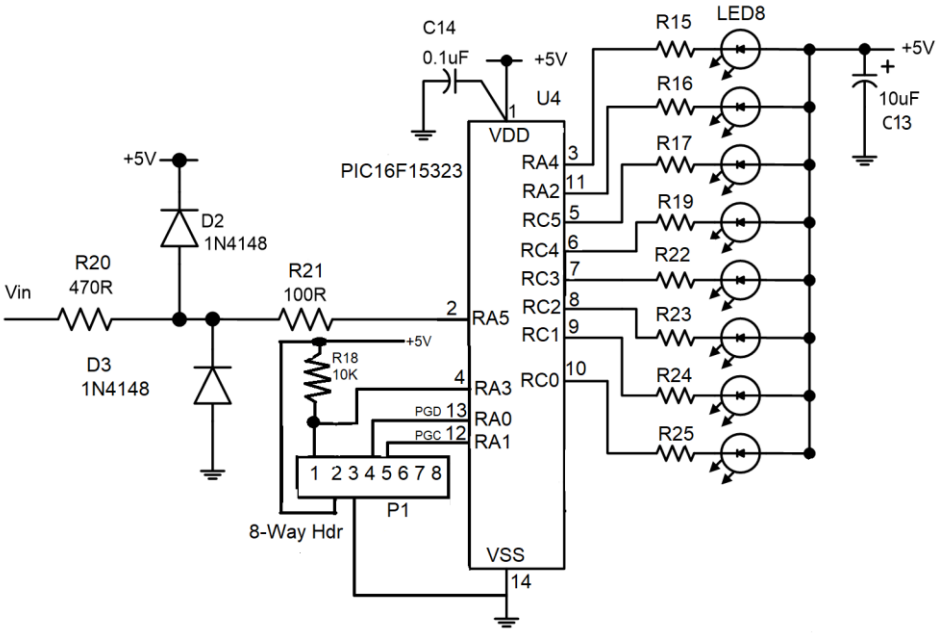


Figure 8: Microcontroller Circuit

8. Programming the PIC microcontroller

PIC microcontrollers come as a blank canvas, and we must program them with code to allow them to function as intended for the sound level meter circuit. To do this please use the following steps.

Step 1: Connect the Programmer

Firstly, make sure that your PIC16f15323 chip is in the IC socket and all of the subcircuits have been tested and are correctly working. **NOTE:** remember to connect the power supply to your circuit correctly before attempting to program.

Next find the programmer on your desk, it will look like Fig. 9.



Figure 9: PIC Programmer

Plug the programmer onto the 8-way header you have fitted, you **MUST** make sure that you attach this the correct way around to prevent any damage to the programmer or your circuit. Make sure the arrow on the programmer is lined up with the white circle printed on the top of the circuit board, as shown in Fig. 10.

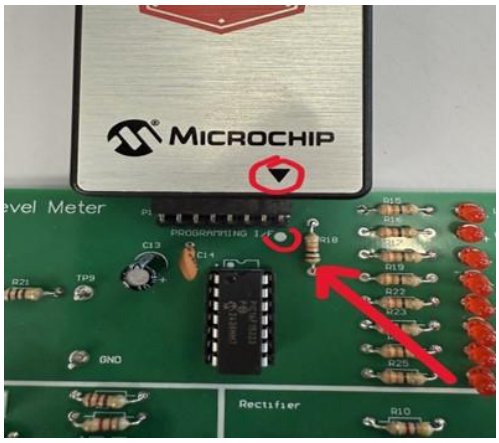


Figure 10: Programmer Orientation

Step 2: Download the Project Code File & Unzip

On Canvas you will be provided with a ZIP file containing the sound level meter code (Sound Level Meterv1.X.zip), you must download this.

Next, open file explorer, go to downloads, and you will see the ZIP file (Fig. 11).

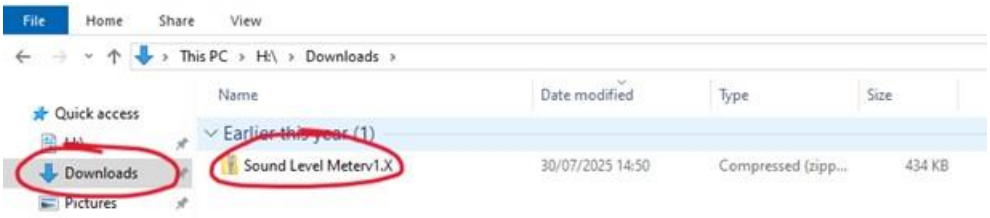


Figure 11: Zip File

Right click on this file, then click 'Extract All' as shown in Fig. 12.

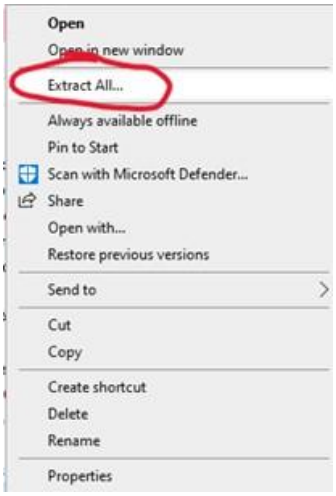


Figure 12: Extract All

This will open another menu and simply click 'Extract' at the bottom

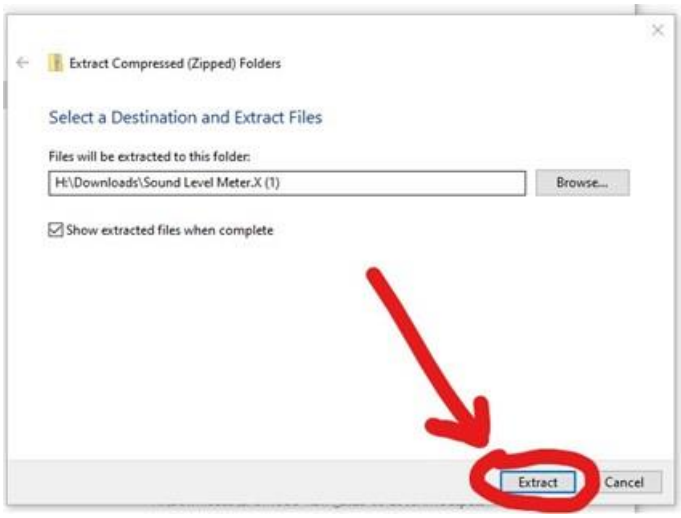


Figure 13: Extract

You should now be left with the unzipped file (highlighted in red in Fig. 14), this is the program we will be using

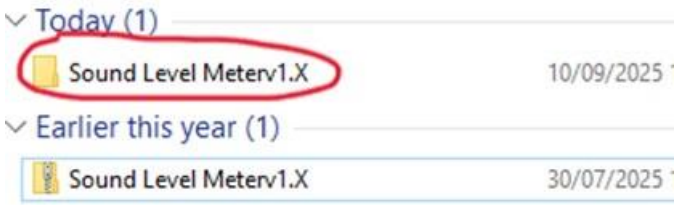


Figure 14: Unzipped File

Step 3: Opening the file in MPLAB X IDE

On the computer search for 'MPLAB X IDE' (Fig. 15)



Figure 15: Open MPLAB

You will then be met with a screen like this:



Figure 16: MPLAB Main Screen

Click ‘File’, then ‘open project’, and locate your unzipped file in downloads. Once you find your unzipped program file, it should look like Fig. 17.



Figure 17: Load Unzipped File

Simply double left click the file to open the program and it will upload all relevant data

Step 4: Uploading the code to the PIC

Finally, we must upload the code we have now opened to the PIC

REMINDER: As previously mentioned, before doing this you must have the correct supplies going to the circuit through the screw terminal, and also make sure the full board is populated with no missing components!

To program the chip, navigate to the tool bar at the top of MPLAB to find the upload button as shown in Fig. 18.

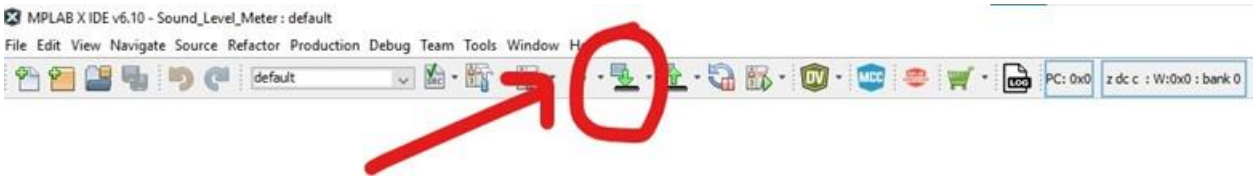


Figure 18: Upload Button

Click this button to fully program your PIC

A list of code should then begin writing, and if it has worked it will end in “Build successful” at the bottom.

Remove the programming unit and move onto the testing part of the project to ensure its functionality

D+

Appendix 1: PIC Programme

```
/**
  Generated Main Source File

  Company:
    Microchip Technology Inc.

  File Name:
    main.c

  Summary:
    This is the main file generated using PIC10 / PIC12 / PIC16 / PIC18 MCUs

  Description:
    This header file provides implementations for driver APIs for all modules selected in the GUI.
    Generation Information :
        Product Revision  :  PIC10 / PIC12 / PIC16 / PIC18 MCUs - 1.81.8
        Device            :  PIC16F15323
        Driver Version    :  2.00
*/

/*
    (c) 2018 Microchip Technology Inc. and its subsidiaries.

    Subject to your compliance with these terms, you may use Microchip software and any
    derivatives exclusively with Microchip products. It is your responsibility to comply with third party
    license terms applicable to your use of third party software (including open source software) that
```

```
    may accompany Microchip software.

    THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER
    EXPRESS, IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY
    IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS
    FOR A PARTICULAR PURPOSE.

    IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE,
    INCIDENTAL OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND
    WHATSOEVER RELATED TO THE SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP
    HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO
    THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL
    CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED THE AMOUNT
    OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS
    SOFTWARE.
*/

#include "mcc_generated_files/mcc.h"

/*
                    Main application
 */
void main(void)
{
    uint8_t count;
    uint16_t convertedValue;       //variable for storing ADC result

    // initialize the device
    SYSTEM_Initialize();
```

```
// When using interrupts, you need to set the Global and Peripheral Interrupt Enable bits
// Use the following macros to:

// Enable the Global Interrupts
//INTERRUPT_GlobalInterruptEnable();

// Enable the Peripheral Interrupts
//INTERRUPT_PeripheralInterruptEnable();

// Disable the Global Interrupts
//INTERRUPT_GlobalInterruptDisable();

// Disable the Peripheral Interrupts
//INTERRUPT_PeripheralInterruptDisable();
   LED1_SetLow();
   LED2_SetHigh();
   LED3_SetLow();
   LED4_SetHigh();
   LED5_SetLow();
   LED6_SetHigh();
   LED7_SetLow();
   LED8_SetHigh();
   __delay_ms(500);
   LED1_SetHigh();
   LED2_SetLow();
   LED3_SetHigh();
   LED4_SetLow();
   LED5_SetHigh();
```

```
    LED6_SetLow();
    LED7_SetHigh();
    LED8_SetLow();
    __delay_ms(500);

    ADC_SelectChannel(5);      //use ADC channel 5 (pin 2)

  while (1)
  {
    // Add your application code
    ADC_StartConversion();                  //read value of potentiometer (ADC range is
between 0 and 1023)
    while(ADC_IsConversionDone());          //wait for result
    convertedValue = ADC_GetConversionResult();    //store value from ADC into variable
    count = convertedValue / 20;     //count = sound level / cal, increment count for every
0.1v analogue input voltage

    if (count > 8)                  //if count > 8 light all LEDs
    count = 8;

  switch (count)                    //switch on count value to illuminate 8 leds
    {
        case 0:
        {PORTA=0b00010100;
         PORTC=0b00111111;
         break;}
        case 1:
        {PORTA=0b00010100;
         PORTC=0b00111110;
         break;}
```

```
        case 2:
        {PORTA=0b00010100;
         PORTC=0b00111100;
         break;}
        case 3:
        {PORTA=0b00010100;
         PORTC=0b00111000;
         break;}
        case 4:
        {PORTA=0b00010100;
         PORTC=0b00110000;
         break;}
case 5:
        {PORTA=0b00010100;
         PORTC=0b00100000;
         break;}
        case 6:
        {PORTA=0b00010100;
         PORTC=0b00000000;
         break;}
        case 7:
        {PORTA=0b00010000;
         PORTC=0b00000000;
         break;}
        case 8:
        {PORTA=0b00000000;
         PORTC=0b00000000;
         break;}
        default:
```

```
        {PORTA=0b00010100;
         PORTC=0b00111111;
        break;}
        }
   }
}
/**
 End of File
*/
```

Appendix 2: PCB.doc

